

Components of a Predictive Parser

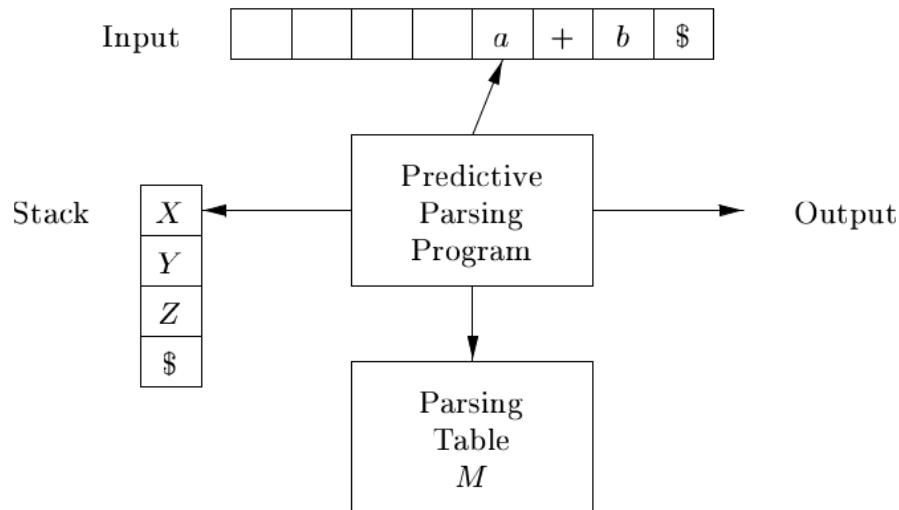


Figure 1: Non-Recursive Predictive Parsing Model

1. Input
 - The input contains the string to be parsed followed by \$.
2. Stack
 - The stack contains a sequence of grammar symbols preceded by \$, the bottom stack marker.
 - Initially the stack contains the start symbol of the grammar.
3. Parsing table
 - The parsing table is a two dimensional array $M[A, a]$ where A is a non-terminal and a is a terminal or the \$ symbol.
4. Output

Construction of the predictive parsing table

Input: Grammar G .

Output: Parsing table M .

Method:

1. For each production $A \rightarrow \alpha$ of the grammar.
 - i. For each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
 - ii. If ϵ is in $FIRST(\alpha)$, then for each terminal b in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, b]$. If ϵ is in $FIRST(\alpha)$ and \$ is in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$ as well.
2. Make each undefined entry of M be error.

Example 1

Create predictive parsing table for a grammar with the following production rules:

1. $E \rightarrow TE'$
2. $E' \rightarrow +TE' \mid \epsilon$
3. $T \rightarrow FT'$
4. $T' \rightarrow *FT' \mid \epsilon$
5. $F \rightarrow (E) \mid id$

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Example 2

Create predictive parsing table for a grammar with the following production rules:

1. $S \rightarrow iEtSS' \mid a$
2. $S' \rightarrow eS \mid \epsilon$
3. $E \rightarrow b$

	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow eS$ $S' \rightarrow \epsilon$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Example 3

Create a predictive parsing table for a grammar with the following production rules:

1. $E \rightarrow TE'$
2. $E' \rightarrow +E \mid \epsilon$
3. $T \rightarrow FT'$
4. $T' \rightarrow T \mid \epsilon$
5. $F \rightarrow PF'$
6. $F' \rightarrow *F' \mid \epsilon$
7. $P \rightarrow (E) \mid a \mid b \mid \epsilon$

[illegible]

	a	b	$($	$)$	$+$	$*$	$\$$
T'	$T' \rightarrow T$	$T' \rightarrow T$	$T' \rightarrow T$	$T' \rightarrow T$ $T \rightarrow \epsilon$	$T' \rightarrow T$ $T' \rightarrow \epsilon$	$T' \rightarrow T$	$T' \rightarrow T$ $T' \rightarrow \epsilon$
F	$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$	$F \rightarrow PF'$
F'	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$	$F' \rightarrow *F'$ $F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$
P	$P \rightarrow a$ $P \rightarrow \epsilon$	$P \rightarrow b$ $P \rightarrow \epsilon$	$P \rightarrow (E)$ $P \rightarrow \epsilon$	$P \rightarrow \epsilon$	$P \rightarrow \epsilon$	$P \rightarrow \epsilon$	$P \rightarrow \epsilon$

LL(1) grammar

- A grammar whose parsing table has no multiple defined entries is said to be LL(1).
- The first L stands for scanning the input from left to right, the second L for producing a left-most derivation and 1 for using one input symbol.

Non-Recursive Predictive Parsing

- The parser is controlled by a program that behaves as follows.
- The program considers X , the symbol on the top of the stack and a , the current input symbol. There are three possibilities:
 1. If $X = a = \$$, the parser halts and announces successful completion of parsing.
 2. If $X = a \neq \$$, the parser pops X off the stack and advances the input pointer to the next input symbol.
 3. If X is a non-terminal, the program consults the entry $M[X, a]$ of the parsing table M . This entry will be either an X -production of the grammar or an *error* entry. If $M[X, a] = \{X \rightarrow UVW\}$, the parser replaces X on the top of the stack by UVW with U on top. As output, we shall assume that the parser just prints the production used. If $M[X, a] = \text{error}$, the parser calls an error recovery routine.

Input: A string w and a parsing table M for grammar G .

Output: If w is in $L(G)$, a left-most derivation of w , otherwise an error indication.

Method:

Initially, input buffer contains $w\$$ where w is the sentence to parse and $\$$ is the input end marker, and the stack contains $\$S$ with S on the top where S is the start symbol of the grammar.

Repeat

Let X be the top stack symbol and a the next input symbol

If X is a terminal or $\$$ then

If $X = a$ then

```

        pop  $X$  from the stack and remove  $a$  from the input
    Else error()
Else /*  $X$  is a non-terminal */
    If  $M[x, a] = X \rightarrow Y_1 Y_2 Y_3 \cdots Y_k$  then
        Begin
            Pop  $X$  from the stack
            Put  $Y_k, Y_{k-1}, Y_{k-2} \cdots Y_1$  onto the stack with  $Y_1$  on the top
            Output production  $X \rightarrow Y_1 Y_2 Y_3 \cdots Y_k$ 
        End
    Else error()
Until  $X = \$$  /* stack is empty */

```

Example 4

Parse $id + id * id$ using the predictive parsing table in example 1.

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Stack	Input	Output
$\$ E$	$id + id * id \$$	$E \rightarrow TE'$
$\$ E' T$	$id + id * id \$$	$T \rightarrow FT'$
$\$ E' T' F$	$id + id * id \$$	$F \rightarrow id$
$\$ E' T' id$	$id + id * id \$$	
$\$ E' T'$	$+ id * id \$$	$T' \rightarrow \epsilon$
$\$ E'$	$+ id * id \$$	$E' \rightarrow +TE'$
$\$ E' T +$	$+ id * id \$$	
$\$ E' T$	$id * id \$$	$T \rightarrow FT'$
$\$ E' T' F$	$id * id \$$	$F \rightarrow id$
$\$ E' T' id$	$id * id \$$	
$\$ E' T'$	$* id \$$	$T' \rightarrow *FT'$
$\$ E' T' F *$	$* id \$$	
$\$ E' T' F$	$id \$$	$F \rightarrow id$
$\$ E' T' id$	$id \$$	
$\$ E' T'$	$\$$	$T' \rightarrow \epsilon$
$\$ E'$	$\$$	$E' \rightarrow \epsilon$
$\$$	$\$$	